COMPUTER ENGINEERING

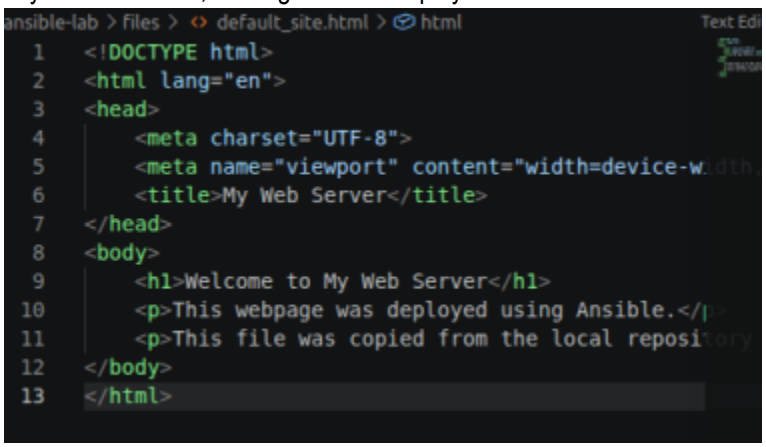
QUEZON CITY

COURSE CODE/TITLE:	CpE212	**SECTION:**	CpE31s2
DATE PERFORMED:		**DATE SUBMITTED:**	
NAME:	Jeus Guille V. Pizarro	**INSTRUCTOR:**	Engr Jimlord Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

What procedures did you perform?
Task 1: Create a file and copy it to remote servers
1. Using the previous directory was created in your previous laboratory, create a directory, and name it “files.” Create a file inside that directory and name it “default_site.html.” Edit the file and put basic HTML syntax. Any content will do, as long as it will display text later. Save the file and exit.



2. Edit the site.yml file and just below the web_servers play, create a new file to copy the default html file for site:
- name: copy default html file for site
tags: apache, apache2, httpd
copy:
src: files/default_site.html
dest: /var/www/html/index.html
owner: root
group: root
mode: '0644'
3. Run the playbook site.yml. Describe the changes.
None

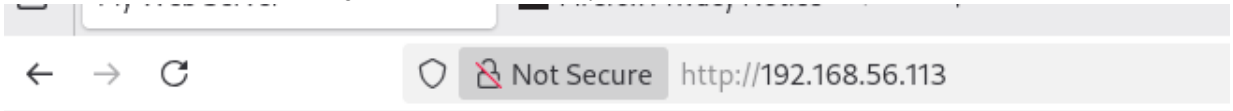
```
ttps://docs.ansible.com/ansible-
ore/2.17/reference_appendices/interpreter_discovery.html for more information.
k: [pizza3]
WARNING]: Platform linux on host pizza_centos is using the discovered Python
nterpreter at /usr/bin/python3.9, but future installation of another Python
nterpreter could change the meaning of that path. See
ttps://docs.ansible.com/ansible-
ore/2.17/reference_appendices/interpreter_discovery.html for more information.
k: [pizza_centos]

ASK [copy default html file for site] ***************************************
k: [pizza2]
k: [pizza3]
k: [pizza_centos]

LAY RECAP *********************************************************************
izza2                      : ok=2    changed=0    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0
izza3                      : ok=2    changed=0    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0
izza_centos                : ok=2    changed=0    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0

izza1@pizza1-local:~/CPE212PIZARRO/ansible-lab$
izza1@pizza1-local:~/CPE212PIZARRO/ansible-lab$ ansible-playb
```

4. Go to the remote servers (web_servers) listed in your inventory. Use cat command to check if the index.html is the same as the local repository file (default_site.html). Do both for Ubuntu and CentOS servers. On the CentOS server, go to the browser and type its IP address. Describe the output.



Welcome to My Web Server

This webpage was deployed using Ansible.

This file was copied from the local repository to the remote server.

5. Sync your local repository with GitHub and describe the changes

.

Task 2: Download a file and extract it to a remote server

1. Edit the site.yml. Just before the web_servers play, create a new play:

- hosts: workstations
become: true
tasks:
- name: install unzip

package:
name: unzip
- name: install terraform
unarchive:
src: https://releases.hashicorp.com/terraform/0.12.28/terraform_0.12.28_linux_amd64.zip
dest: /usr/local/bin
remote_src: yes
mode: 0755
owner: root
group: root

```
---
- hosts: workstations
  become: true
  tasks:
  - name: install unzip
    package:
      name: unzip
  - name: install terraform
    unarchive:
      src: https://releases.hashicorp.com/terrafo
      dest: /usr/local/bin
      remote_src: yes
      mode: 0755
      owner: root
      group: root
```

note: In Ansible, a play is a section of a playbook that defines which remote machines Ansible should manage and what tasks it should perform on them.
2. Edit the inventory file and add a workstation group. Add any Ubuntu remote server. Make sure to remember the IP address.
3. Run the playbook. Describe the output.

```
TASK [copy default html file for site] ***************************************
ok: [pizza2]
ok: [pizza3]
ok: [pizza_centos]

PLAY RECAP *******************************************************************
pizza2                     : ok=2    changed=0    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0
pizza3                     : ok=2    changed=0    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0
pizza_centos               : ok=2    changed=0    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0
workstation1               : ok=3    changed=1    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0

pizza1@pizza1-local:~/CPE212PIZARRO/ansible-lab$
pizza1@pizza1-local:~/CPE212PIZARRO/ansible-lab$ ansible-playb
```

4. On the Ubuntu remote workstation, type terraform to verify installation of terraform. Describe the output.

```
pizza2@server1:~$ terraform
Usage: terraform [-version] [-help] <command> [args]

The available commands for execution are listed below.
The most common, useful commands are shown first, followed by
less common or more advanced commands. If you're just getting
started with Terraform, stick with the common commands. For the
other commands, please read the help and docs before usage.

Common commands:
    apply              Builds or changes infrastructure
    console            Interactive console for Terraform interpolations
    destroy            Destroy Terraform-managed infrastructure
    env                Workspace management
    fmt                Rewrites config files to canonical format
    get                Download and install modules for the configuration
    graph              Create a visual graph of Terraform resources
    import             Import existing infrastructure into Terraform
    init               Initialize a Terraform working directory
    login              Obtain and save credentials for a remote host
    logout             Remove locally-stored credentials for a remote host
```

Task 3: Create roles

1. Edit the site.yml. Configure roles as follows: (make sure to create a copy of the old site.yml file because you will be copying the specific plays for all groups)

Save the file and exit.

2. Under the same directory, create a new directory and name it roles. Enter the roles directory and create new directories: base, web_servers, file_servers, db_servers and workstations. For each directory, create a directory and name it tasks.

```
pizza1@pizza1-local:~/CPE212PIZARRO/ansible-lab$ mkdir -p base/tasks
pizza1@pizza1-local:~/CPE212PIZARRO/ansible-lab$ mkdir -p web_servers/tasks
pizza1@pizza1-local:~/CPE212PIZARRO/ansible-lab$ mkdir -p db_servers/tasks
pizza1@pizza1-local:~/CPE212PIZARRO/ansible-lab$ mkdir -p workstation_servers/ta
sks
pizza1@pizza1-local:~/CPE212PIZARRO/ansible-lab$ mkdir -p workstation/tasks
pizza1@pizza1-local:~/CPE212PIZARRO/ansible-lab$
pizza1@pizza1-local:~/CPE212PIZARRO/ansible-lab$ ansible-playb
```

3. Go to tasks for all directory and create a file. Name it main.yml. In each of the tasks for all directories, copy and paste the code from the old site.yml file. Show all contents of main.yml files for all tasks.

4. Run the site.yml playbook and describe the output.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Answer the following:

1. What is the importance of creating roles?

Creating roles structures complex automation projects into modular, reusable directory components that separate tasks, files, templates, and variables[cite: 18]. This prevents monolithic playbook files, streamlines collaborative development, and allows configuration units to be shared across multiple environments

2. What is the importance of managing files?

Automating file operations with modules like `copy`, `unarchive`, and `file` ensures configuration consistency and accurate content deployment across remote nodes[cite: 18]. It eliminates manual error during software

configuration, enforces proper file permissions and ownership, and guarantees repeatable deployments without individual SSH logins

Conclusions/Learnings:
Through this activity, I gained hands-on experience in automating configuration management using Ansible. I learned how to deploy files across remote nodes using the copy module, manage remote archives and dependencies with the unarchive module, and organize complex playbooks into modular, reusable Ansible roles. This emphasized the importance of Infrastructure as Code (IaC) in improving deployment efficiency, consistency, and scalability.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.